

Продукты



Решения



Разработчики



Партнеры



Цены

Вход ▾

Регистрация ▾

Обучающие материалы

Вопросы

Документация по продукту

Поиск

Обучающие материалы / Искусственный интеллект и машинное обучение /
Управление затратами на мультимодальные API с помощью Inference Router

Учебник

Управление затратами на мультимодельный API с помощью Inference Router

Опубликовано 23 июня 2026 года



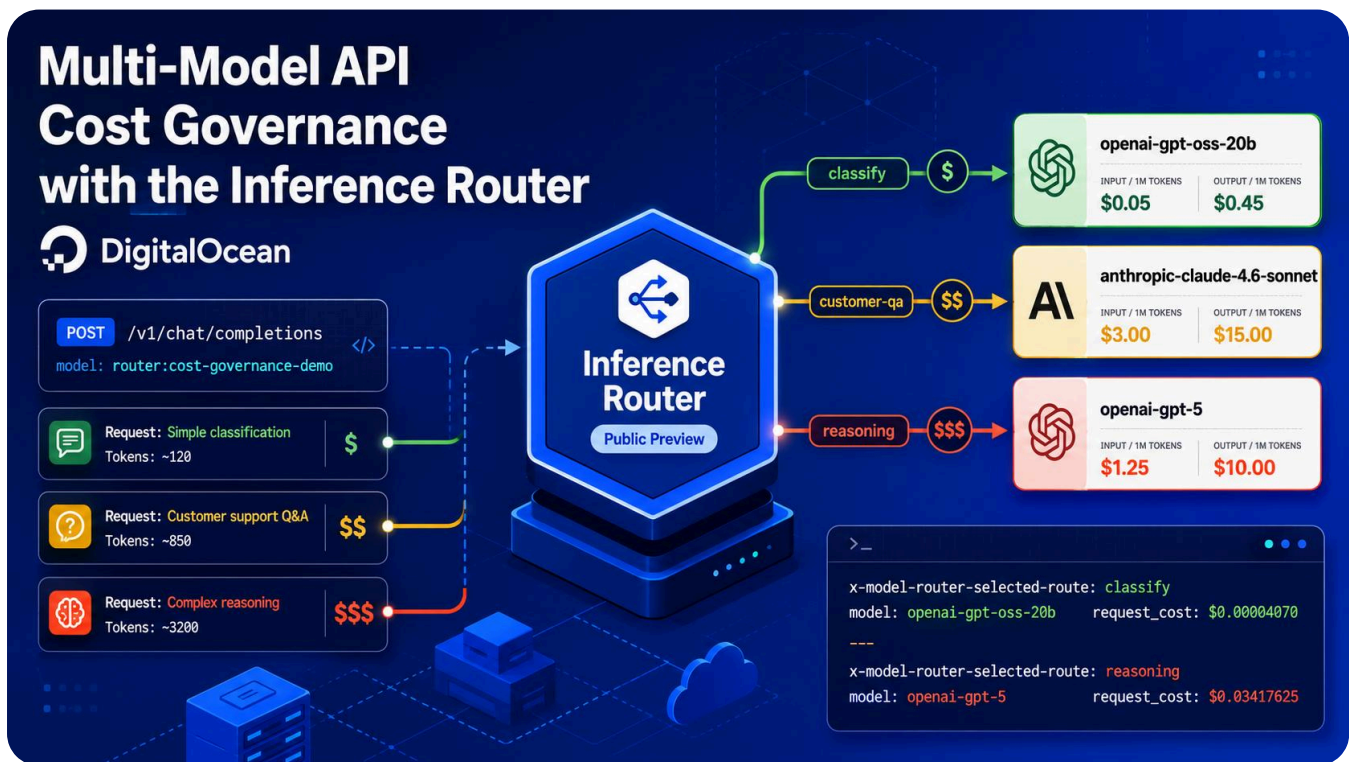
AI/МЛ

Вывод искусственного интеллекта

Выделенный вывод



Автор: [Винаяк Баранвал](#)
Технический писатель II



Содержание

Популярные темы

Введение

Маршрутизатор логического вывода — это промежуточное программное обеспечение, которое находится между вашим приложением и уровнем обслуживания моделей. Оно направляет каждый вызов API больших языковых моделей к модели, подходящей для решения конкретной задачи, а не отправляет все запросы на одну и ту же конечную точку. Проблема, которую он решает, связана с биллингом, который по умолчанию используется в большинстве бэкендов SaaS: когда каждый запрос обрабатывается одной моделью, за простые вызовы классификации с 94 входными токенами и сложные вызовы логического вывода с 3411 выходными токенами взимается плата, установленная для более сложной задачи. Дешевая задача субсидирует дорогую, но в неправильном направлении.

Маршрутизатор логического вывода DigitalOcean, являющийся частью [Inference Engine](#), запущенного в апреле 2026 года, в настоящее время находится на стадии публичного тестирования.

В этом руководстве мы создадим работающий маршрутизатор с тремя политиками задач для бэкенда SaaS-поддержки: низкозатратный путь с классификатором, путь с вопросами и ответами для клиентов, чувствительный к качеству, и логический путь. В итоге у вас будет маршрутизатор, который можно вызывать через

стандартную конечную точку OpenAI для завершения диалогов, сигналы о стоимости каждого запроса, которые можно прочитать из заголовка ответа, и шаблон закрепления сеанса для пути с вопросами и ответами, который поддерживает KV-кэш в активном состоянии во время многоэтапных диалогов.

Ключевые рекомендации на Вынос

- Серверная часть SaaS, которая направляет каждый запрос через одну фронтенд-модель, платит по фронтенд-тарифам за вызовы классификации, с которыми одинаково хорошо справляется модель с открытым исходным кодом, имеющая 20 миллиардов параметров, — до 36 раз дороже за каждый запрос, согласно подтвержденным данным.
- Цифровой маршрутизатор логического вывода DigitalOcean использует классификатор «смесь экспертов» (MoE) для сопоставления каждого входящего запроса с описаниями задач на естественном языке, которые вы настраиваете. Затем политика выбора для сопоставленной задачи выбирает конкретную модель из пула моделей для этой задачи. Оценка упорядоченных правил не предусмотрена.
- Требуются два отдельных идентификатора, и оба они должны принадлежать одной команде DigitalOcean: персональный токен доступа (`$DIGITALOCEAN_TOKEN`) для плоскости управления, которая создает маршрутизаторы и управляет ими, и ключ доступа к модели (`$MODEL_ACCESS_KEY`) с префиксом `sk-do-` для вызова логического вывода. Несоответствие команд приводит к сбою вызова, даже если оба идентификатора действительны по отдельности.
- Для ранжирования вручную не требуется поле `selection_policy` в теле запроса на создание. Порядок в списке `models[]` — это выражение ранжирования, подтвержденное с помощью вызова API в реальном времени 16 июня 2026 года.
- Во время публичного тестирования сам маршрутизатор предоставляется бесплатно. Вы платите только за модели, которые обслуживают каждый запрос. Вызов маршрутизатора добавляет небольшую задержку в несколько секунд на каждый запрос.

Prerequisites

Before following this tutorial, you need:

- A DigitalOcean account at **Tier 3 or higher**. Claude Sonnet 4.6 and GPT-5 are commercial models that require Tier 3+ access. Tier 1 and Tier 2 accounts are limited to open-source models. [Check your tier and limits](#).
- A DigitalOcean Personal Access Token (PAT) with write access to the Inference API, stored as `$DIGITALOCEAN_TOKEN` . This credential is for the control plane only: creating and managing routers. Do not use it for inference calls.
- A Model Access Key (MAK) from the Inference console, stored as `$MODEL_ACCESS_KEY` . This credential carries an `sk-do-` prefix and is used exclusively for inference invocation.
- Both the PAT and the MAK **must belong to the same DigitalOcean team**. A team mismatch causes invocation failures regardless of how the router is configured. This is the most common setup error when working across multiple DO accounts.
- `curl` for the control-plane calls in this tutorial, and Python with the `openai` package (`pip install openai`) for the invocation examples.
- Familiarity with the OpenAI chat completions API format.

Model slugs used in this tutorial: `openai-gpt-oss-20b` , `llama3.3-70b-instruct` , `anthropic-claude-4.6-sonnet` , `openai-gpt-5` . Verify current availability in the [model catalog](#).

What Is an Inference Router?



An inference router is a middleware component that receives LLM API requests and directs each one to an appropriate model based on the task type and configured selection policy, without any routing logic in your application code. Your application sends every request to a single endpoint with `"model": "router:<your-router-name>"` , and the router handles dispatch.

How the Inference Router Fits into the DigitalOcean Inference Engine

The [Inference Engine](#) bundles serverless inference, dedicated inference, and the Inference Router under a unified API surface. The Router is the layer that governs which model within the Inference Engine serves each incoming request. It is invoked at the inference endpoint (`https://inference.do-ai.run/v1/chat/completions`) using the standard OpenAI chat completions format. The [Inference Router how-to](#) documents the full parameter set for router creation and management. The [available models list](#) shows current model slugs and which tiers can access them.

Serverless Inference vs. Dedicated Inference: What Gets Routed and Why

Serverless inference runs on shared infrastructure with per-request billing and automatic scaling, including scale-to-zero. Dedicated inference runs on reserved compute with predictable latency and fixed capacity per endpoint. The Inference Router can target both. For workloads requiring guaranteed capacity and consistent latency SLAs, the Speed Optimization or Manual Ranking selection policies let you designate a dedicated model as the first choice in a task pool. See [serverless vs. dedicated vs. batch inference](#) for the capacity and cost tradeoffs between deployment types.

How Inference Routing Works ^

The Inference Router is a semantic router, not a rule-based dispatcher. Each incoming prompt is evaluated by an MoE classifier model, which matches it against the natural-language `custom_task.description` fields you define when creating the router. The matched task's selection policy then picks a specific model from that task's pool. There is no ordered evaluation and no "first matching task wins" behavior.

Task Matching and Selection Policies

Task matching is semantic: the MoE classifier reads the full prompt and matches it to the closest task description. The quality of your task descriptions determines match accuracy. Write `custom_task.description` values as descriptive task definitions, not as labels or category names.

The Router supports four selection policies:

Policy	API Expression	Use When
Cost Efficiency	"selection_policy": { "prefer": "cheapest" }	Multi-model pool; minimize spend per request
Speed Optimization	"selection_policy": { "prefer": "fastest" }	Multi-model pool; minimize time-to-first-token

Policy	API Expression	Use When
Manual Ranking	No <code>selection_policy</code> field. List order in <code>models[]</code> is the ranking expression.	Quality-sensitive path requiring deterministic model preference
Optimal	<code>"task_slug": "<preset>"</code>	DO-defined preset task types only; not available for custom tasks

For Manual Ranking, omitting the `selection_policy` field entirely is the correct API expression. This was confirmed via a live create call on June 16, 2026: the API echoes models back in list order, and that order is the policy.

Routing Approaches: Static, Semantic, and Cost-Based

Three distinct approaches describe how inference routers dispatch requests:

Static routing dispatches based on fixed attributes of the request, such as URL path, a header value, or a request field. It requires no classifier and adds minimal latency, but it cannot adapt to prompt content. A billing question and a bug report sent to the same endpoint look identical from a static router’s perspective. Use this approach when workloads are already segmented by application logic or explicit model parameters.

Semantic routing evaluates the content of the prompt to determine task type before dispatching. The DigitalOcean Inference Router uses this approach through its MoE classifier. The classifier adds a small, sub-second routing overhead, but it enables dispatch decisions based on actual task content rather than a proxy signal. Research from EMNLP 2024 found that prompt-content-based routing can improve query efficiency by roughly 40%, reduce cost by 30%, and improve output quality by 10%, depending on workload composition (Stripelis et al., [TensorOpera Router: A Multi-Model Router for Efficient LLM Inference](#)).

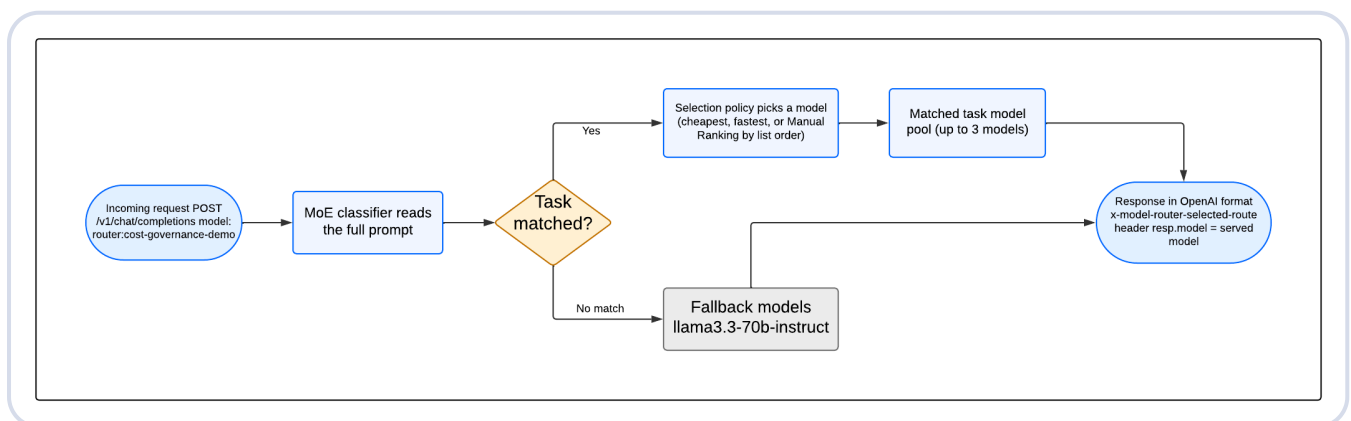
Cost-aware dynamic routing extends semantic routing with real-time pricing signals, current model availability, or quality scores to select models dynamically based on current conditions rather than static configuration. This is more complex to implement and maintain but relevant for backends that span multiple providers or need to respond to per-model pricing changes without router reconfiguration.

Open-source alternatives for teams building outside DigitalOcean include the [vLLM Semantic Router](#), which uses signal-driven semantic classification to route requests across model pools, and [llm-d-router](#), which provides KV-cache and load-aware routing for self-hosted Kubernetes-based serving stacks.

Request Flow from API Call to Model Response

When your application sends a request to `https://inference.dog.ai.run/v1/chat/completions` with `"model": "router:cost-governance-demo"` :

1. The Inference Router passes the prompt to its MoE classifier.
2. The classifier matches the prompt against your configured `custom_task.description` values and returns the closest task.
3. If a task matches, the task's selection policy picks a model from that task's `models[]` pool and forwards the request.
4. If no task matches, or the matched model is unavailable, the request goes to `fallback_models[]` .
5. The response returns in standard OpenAI chat completions format, with the addition of the `x-model-router-selected-route` response header identifying which task matched.



The `model` field in the response body shows which model served the request. These two fields together give you per-request routing attribution without opening the Analyze dashboard.

The Cost Problem This Solves

As benchmarked in [Metrics that Matter with Serverless Inference](#), the cost of a single completed answer swings roughly 230 times across the model catalog, driven almost entirely by model choice, not provider pricing differences.

The generalization tax is what you pay when a backend uses one frontier model for all tasks. At the token counts confirmed in the live runs for this tutorial, a single classification call (94 in / 80 out) costs \$0.00004070 on `openai-gpt-oss-20b` . The same call sent to Claude Sonnet 4.6 costs \$0.00148200, a 36x premium. Sent to GPT-5, it costs \$0.00091750, a 22.5x premium. Those multiples apply to every classification request in your traffic volume.

At 700,000 classification requests per month, the difference between routing classify traffic to `openai-gpt-oss-20b` and hardcoding Claude Sonnet 4.6 is \$28.49 vs. \$1,037.40 per month. The routing architecture below captures that saving without changing a line of application code.

Architecture: Three Paths, Three Model Tiers ^

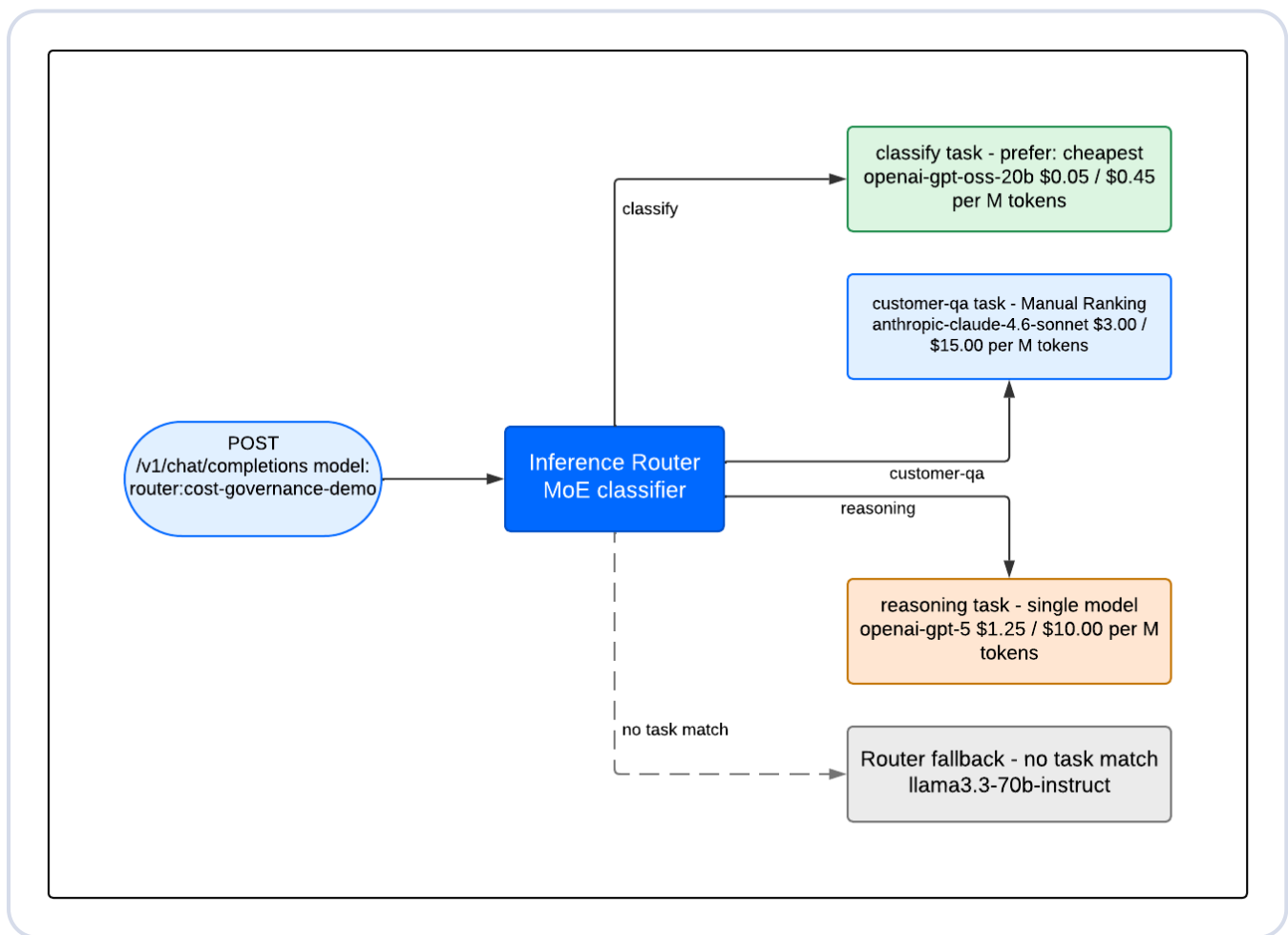
This tutorial implements a three-path router reflecting the task-complexity structure of a SaaS support backend.

Classifier path. Incoming support tickets are first classified into categories: billing, bug, how-to, or account. This is a short-input, categorical-output task. `openai-gpt-oss-20b` is the primary model because it is the cheapest option in the pool and produces correct categorical labels on this task type. `llama3.3-70b-instruct` is the fallback. Selection policy: `prefer: cheapest` .

Customer Q&A path. Multi-turn, user-facing questions require consistent quality across sessions and reliable behavior on domain-specific content. `anthropic-claude-4.6-sonnet` is the primary model, `llama3.3-70b-instruct` is the fallback. Selection policy: Manual Ranking (Sonnet listed first, `selection_policy` field omitted). The reasoning for Manual Ranking over `prefer: fastest` is that the fastest model in the pool could be the weaker one on any given run, and this path is quality-sensitive. Manual Ranking gives deterministic Sonnet-unless-unavailable behavior.

Reasoning path. Complex multi-step reasoning tasks justify GPT-5's per-request cost because the task value is high. Note that GPT-5's input rate (\$1.25/M) is lower than Claude Sonnet 4.6's (\$3.00/M). The reasoning path costs more because it generates substantially more output tokens (3,411 in the confirmed live run vs. 292 for Q&A), and output tokens dominate cost on reasoning paths regardless of headline rate.

Router fallback. `llama3.3-70b-instruct` catches prompts that the MoE classifier does not match to any configured task. This prevents unmatched requests from returning an error and routes them to a capable open-source model at low cost.



Before configuring task tiers for your own backend, measure your actual call distribution. A backend where 80% of traffic is reasoning requests will see a different cost profile than one where 80% is classification. Run a quality evaluation on your task categories using Router Evaluation in the Playground to confirm that cheaper models produce acceptable output before routing live traffic to them.

Setting Up the Inference Router

Creating the Router via the API

Send a `POST` request to the control-plane endpoint using your PAT. The request below creates all three task policies in a single call:

Copy

```
curl -s -X POST "https://api.digitalocean.com/v2/gen-ai/models/routers" \
-H "Authorization: Bearer $DIGITALOCEAN_TOKEN" \
-H "Content-Type: application/json" \
-d '{
```

```

"name": "cost-governance-demo",
"description": "Three-path router for SaaS support: classify, Q-and-A, re
"policies": [
  {
    "custom_task": {
      "name": "classify",
      "description": "Classify a customer support message into exactly one of the following categories: product, pricing, general, or other. Return the category name as a JSON object with the key 'category'."
    },
    "models": ["openai-gpt-oss-20b", "llama3.3-70b-instruct"],
    "selection_policy": { "prefer": "cheapest" }
  },
  {
    "custom_task": {
      "name": "customer-qa",
      "description": "Answer a customer-facing question about the product or service. Provide a clear, concise, and helpful response."
    },
    "models": ["anthropic-claude-4.6-sonnet", "llama3.3-70b-instruct"]
  },
  {
    "custom_task": {
      "name": "reasoning",
      "description": "Perform complex multi-step reasoning, mathematical calculations, or logical deductions. Provide a step-by-step explanation of your reasoning."
    },
    "models": ["openai-gpt-5"]
  }
],
"fallback_models": ["llama3.3-70b-instruct"]
}'

```

Three notes on this body:

The `customer-qa` policy has no `selection_policy` field. That is correct and intentional. Omitting the field activates Manual Ranking: the Router tries models in list order. Claude Sonnet 4.6 is listed first and is tried first.

The `custom_task.description` values are inputs to the MoE classifier model. Write them as task definitions, not display labels. Vague descriptions like “general questions” overlap with most incoming prompts and degrade routing accuracy.

The PAT (`$DIGITALOCEAN_TOKEN`) goes in this request. The MAK (`$MODEL_ACCESS_KEY`) is not used here.

Output

```

{
  "model_router": {
    "uuid": "11f16981-ed77-8bd0-ae4-4e013e2ddde4",
    "name": "cost-governance-demo",

```

```

    "description": "Three-path router for SaaS support: classify, Q-and-A, re
    "regions": ["all"],
    "config": {
      "policies": [
        {
          "custom_task": {
            "name": "classify",
            "description": "Classify a customer support message into exactly
          },
          "models": ["openai-gpt-oss-20b", "llama3.3-70b-instruct"],
          "selection_policy": { "prefer": "cheapest" }
        },
        {
          "custom_task": {
            "name": "customer-qa",
            "description": "Answer a customer-facing question about the produ
          },
          "models": ["anthropic-claude-4.6-sonnet", "llama3.3-70b-instruct"]
        },
        {
          "custom_task": {
            "name": "reasoning",
            "description": "Perform complex multi-step reasoning, mathematica
          },
          "models": ["openai-gpt-5"]
        }
      ],
      "fallback_models": ["llama3.3-70b-instruct"]
    },
    "created_at": "2026-06-16T12:50:18Z",
    "updated_at": "2026-06-16T12:50:18Z"
  }
}

```

Confirm three things in the response: `uuid` is present (save it if you need it for API-based edits or cleanup later), the `customer-qa` policy body shows no `selection_policy` field, and `fallback_models` contains `llama3.3-70b-instruct`. If you receive an HTTP 400 with `"model router name already exists"`, the name `cost-governance-demo` is already taken in your team. Use a different name.

Creating the Router via the Control Panel

The [DigitalOcean console](#) provides a visual router creation flow at **AI/ML > Inference > My Routers > Create Router**. For each task policy, you specify the task name, task description, model pool, and selection policy in form fields. The control panel does not require `curl` or a PAT.

For the Manual Ranking policy on the Q&A path, leave the selection policy dropdown unset. Model order in the pool list determines ranking. The router is immediately invocable by name after creation.

Routers created through either the API or the console are editable afterward from the **My Routers** menu using **Edit Router**, up to 3 models per task pool.

Verifying Routing Behavior

To confirm the router was created and is queryable, retrieve it by the UUID returned in the create response:

Copy

```
curl -s "https://api.digitalocean.com/v2/gen-ai/models/routers/11f16981-ed77-  
-H "Authorization: Bearer $DIGITALOCEAN_TOKEN"
```

Output

```
{  
  "model_router": {  
    "name": "cost-governance-demo",  
    "config": {  
      "policies": [  
        { "custom_task": { "name": "classify" }, "models": ["openai-gpt-oss-2"] },  
        { "custom_task": { "name": "customer-qa" }, "models": ["anthropic-claude-3.5"] },  
        { "custom_task": { "name": "reasoning" }, "models": ["openai-gpt-5"] }  
      ],  
      "fallback_models": ["llama3.3-70b-instruct"]  
    }  
  }  
}
```

A 200 response with the full configuration confirms the router is registered and queryable. If you receive a 404, the UUID does not match a router in your team, or the PAT belongs to a different team than the one where the router was created.

Invoking the Router from Your Backend



All inference calls use the MAK (`$MODEL_ACCESS_KEY`), not the PAT. The model field is `"model": "router:<your-router-name>"`.

The three requests below demonstrate one prompt per task path using `curl`.

Classifier path:

[Copy](#)

```
curl -s "https://inference.do-ai.run/v1/chat/completions" \
-H "Authorization: Bearer $MODEL_ACCESS_KEY" \
-H "Content-Type: application/json" \
-d '{
  "model": "router:cost-governance-demo",
  "messages": [
    {
      "role": "user",
      "content": "Classify this support message into one of: billing, bug,
    }
  ]
}'
```

Output

```
{
  "id": "chatcmpl-...",
  "object": "chat.completion",
  "model": "openai-gpt-oss-20b",
  "choices": [
    {
      "message": { "role": "assistant", "content": "billing" },
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 94,
    "completion_tokens": 80,
    "total_tokens": 174
  }
}
```

"model": "openai-gpt-oss-20b" confirms `prefer: cheapest` selected the lower-cost model. Token counts of `94 in / 80 out` match the verified live run from June 16, 2026. The `completion_tokens: 80` count for a one-word answer is expected: `openai-gpt-oss-20b` is a reasoning-style open model that emits internal reasoning tokens counted as completion tokens before outputting the visible label. The answer `"billing"` is the visible output; the remaining tokens are the model's internal process.

Customer Q&A path:

Copy

```
curl -s "https://inference.do-ai.run/v1/chat/completions" \
-H "Authorization: Bearer $MODEL_ACCESS_KEY" \
-H "Content-Type: application/json" \
-d '{
  "model": "router:cost-governance-demo",
  "messages": [
    {
      "role": "user",
      "content": "How do I reset my password if I no longer have access to
    },
    ],
  "max_completion_tokens": 512
}'
```

Output

```
{
  "id": "chatcmpl-...",
  "object": "chat.completion",
  "model": "anthropic-claude-4.6-sonnet",
  "choices": [
    {
      "message": {
        "role": "assistant",
        "content": "To reset your password without access to your registered
      },
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 24,
    "completion_tokens": 292,
    "total_tokens": 316
  }
}
```

"model": "anthropic-claude-4.6-sonnet" confirms Manual Ranking sent the request to the first-listed model. Token counts 24 in / 292 out match the June 16 verified run.

Reasoning path:


```
curl -s "https://inference.do-ai.run/v1/chat/completions" \
-H "Authorization: Bearer $MODEL_ACCESS_KEY" \
-H "Content-Type: application/json" \
-d '{
  "model": "router:cost-governance-demo",
  "messages": [
    {
      "role": "user",
      "content": "A service has three dependencies with 99.9%, 99.95%, and
    }
  ],
  "max_completion_tokens": 4096
}'
```

Output

```
{
  "id": "chatcmpl-...",
  "object": "chat.completion",
  "model": "openai-gpt-5",
  "choices": [
    {
      "message": {
        "role": "assistant",
        "content": "Combined availability is calculated by multiplying the in
      },
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 53,
    "completion_tokens": 3411,
    "total_tokens": 3464
  }
}
```

"model": "openai-gpt-5" confirms the reasoning task matched and GPT-5 served it. The token count 53 in / 3,411 out matches the June 16 verified run. The max_completion_tokens here is set to 4096 . Setting it to 1024 on this same reasoning prompt returned "content": null with "finish_reason": "length" in the confirmed live test: GPT-5 consumed the full budget on internal reasoning steps and had no tokens left for the visible answer. Set a minimum budget of 4,096 tokens on any reasoning path, and adjust upward for longer expected outputs.

GPT-5 works through the Inference Router over `/v1/chat/completions` without any special handling on your end. The Router manages the dispatch transparently. However, GPT-5 may answer correctly while declining to expose internal reasoning steps in the response body. Do not write application logic that depends on chain-of-thought output being present.

The [Python](#) equivalent for the classifier path using the OpenAI SDK:

Copy

```
import os
from openai import OpenAI

client = OpenAI(
    api_key=os.environ["MODEL_ACCESS_KEY"],
    base_url="https://inference.do-ai.run/v1/"
)

response = client.chat.completions.create(
    model="router:cost-governance-demo",
    messages=[
        {
            "role": "user",
            "content": "Classify this support message into one of: billing, b
        }
    ]
)

print(response.choices[0].message.content)
print(f"Served by: {response.model}")
```

Output

```
billing
Served by: openai-gpt-oss-20b
```

`response.model` shows which model served the request. This is your per-request cost audit signal without opening the dashboard.

Reading the Cost Signal per Request



The `x-model-router-selected-route` response header identifies which task policy matched. `response.model` identifies which model served. Together, they give you per-request routing attribution.

To read the response header in Python, use `.with_raw_response` :

[Copy](#)

```
import os
from openai import OpenAI

client = OpenAI(
    api_key=os.environ["MODEL_ACCESS_KEY"],
    base_url="https://inference.do-ai.run/v1/"
)

raw = client.chat.completions.with_raw_response.create(
    model="router:cost-governance-demo",
    messages=[
        {
            "role": "user",
            "content": "Classify this support message into one of: billing, b
        }
    ]
)

response = raw.parse()
matched_task = raw.headers.get("x-model-router-selected-route")
serving_model = response.model
usage = response.usage

print(f"Matched task:    {matched_task}")
print(f"Served by:       {serving_model}")
print(f"Tokens in/out:    {usage.prompt_tokens} / {usage.completion_tokens}")
```

Output

```
Matched task:    classify
Served by:       openai-gpt-oss-20b
Tokens in/out:   94 / 80
```

`x-model-router-selected-route: classify` confirms the prompt matched the classifier task, not the fallback. If this header returns `fallback` instead of a named task, the

prompt did not match any configured task description. That is the primary signal for routing mismatches and triggers the debugging steps in the Observability section.

Multi-Model API Cost Governance ^

Mapping Tasks to Cost Outcomes

Per-request cost for each path, using token counts from the June 16, 2026 live runs and prices verified in May 2026:

Path	Model	Tokens (in/out)	Per-request cost
Classify	openai-gpt-oss-20b	94 / 80	\$0.00004070
Customer Q&A	anthropic-claude-4.6-sonnet	24 / 292	\$0.00445200
Reasoning	openai-gpt-5	53 / 3,411	\$0.03417625

Cost arithmetic (verified):

- Classify: $(94 \times \$0.05 + 80 \times \$0.45) / 1,000,000 = \textbf{\$0.00004070}$
- Q&A: $(24 \times \$3.00 + 292 \times \$15.00) / 1,000,000 = \textbf{\$0.00445200}$
- Reasoning: $(53 \times \$1.25 + 3,411 \times \$10.00) / 1,000,000 = \textbf{\$0.03417625}$

All prices are subject to change. Current pricing at <https://docs.digitalocean.com/products/inference/details/pricing/>.

Using Model Tiers to Control Inference Spend

The reasoning path costs approximately 840 times more per request than the classifier path (\$0.03417625 vs. \$0.00004070). Routing requests to appropriate tiers based on task complexity keeps average cost proportional to task value.

The GPT-5 cost structure is worth understanding precisely: GPT-5’s input rate (\$1.25/M) is lower than Claude Sonnet 4.6’s (\$3.00/M). The reasoning path costs more than the Q&A path because reasoning generates approximately 12 times more output tokens (3,411 vs. 292 in the live runs), and GPT-5’s output rate (\$10.00/M) applies to those tokens. Cost on reasoning paths is dominated by output token volume. This is the same dynamic documented in the 230x cost-per-answer spread in the [Metrics that Matter with Serverless Inference](#) benchmark: reasoning models generate thinking tokens billed as output before the visible answer begins.

Cost Comparison Table: Routed vs. Hardcoded Frontier

This table uses a traffic split of 700,000 classify requests, 250,000 Q&A requests, and 50,000 reasoning requests per month, based on the per-request costs confirmed in the June 16, 2026 live runs:

Configuration	Classify/month	Q&A/month	Reasoning/month	Total/month
Routed (tiered)	\$28.49	\$1,113.00	\$1,708.81	\$2,850.30
Hardcode Claude Sonnet 4.6	\$1,037.40	\$1,113.00	\$2,566.20	\$4,716.60
Hardcode Claude Opus 4.7	\$1,729.00	\$1,855.00	\$4,277.00	\$7,861.00
Hardcode GPT-5	\$642.25	\$737.50	\$1,708.81	\$3,088.56

Against a Claude Sonnet 4.6 baseline, routing saves 39.6% per month (\$1,866.30 at this traffic volume). Against Claude Opus 4.7, savings are 63.7% (\$5,010.70). Against GPT-5 as the hardcoded model, savings are 7.7% (\$238.26), because the Q&A path routed to Sonnet (\$1,113.00) costs more than Q&A hardcoded on GPT-5 (\$737.50). The savings against GPT-5 come almost entirely from the classifier path.

The 39.6% and 63.7% savings are sensitive to traffic composition. A backend where reasoning traffic constitutes a larger share of requests will see smaller savings percentages, because the routing decision on the reasoning path is a single-model pool: the Router adds overhead without offering a cheaper alternative on that path. The savings come from the classify and Q&A tier separation.

Multi-Model Orchestration Patterns

Pattern 1: Complexity-Based Routing

The three-path router built in this tutorial is complexity-based routing: tasks are segmented by the cognitive complexity of the required output, and each tier uses a model sized for that task. The classifier path uses a 20B-parameter model, the Q&A path uses a frontier chat model, and the reasoning path uses a reasoning-optimized model.

This pattern fits backends where task types are distinct and separable by prompt content. If your backend handles document summarization, entity extraction, and code generation as distinct call types, define three task policies with descriptions that reflect those output types, and the MoE classifier will route accordingly. Keep each task description specific to its output format, because the classifier performs better when task descriptions describe distinct output types rather than overlapping topic areas.

Pattern 2: Fallback for Availability and Cost Guardrails

The `fallback_models` field provides a safety net for unmatched prompts. In the router created above, `llama3.3-70b-instruct` catches any prompt the classifier does not match to a named task. This also serves as a cost guardrail: unclassified requests route to the open-source fallback rather than silently hitting a commercial frontier model.

The Q&A policy's Manual Ranking structure provides a second fallback layer within the policy itself. If Claude Sonnet 4.6 is unavailable, the Router falls back to `llama3.3-70b-instruct` before reaching the global `fallback_models` array. This gives the Q&A path graceful degradation without a total outage.

For backends that need strict per-request cost caps, one pattern is to add a budget-checking layer in application middleware before the router call. If the estimated cost of a reasoning request exceeds a threshold, the application reframes the request as a Q&A task or declines it before it reaches the Router. The Router itself does not enforce per-request spend limits.

Pattern 3: Session Pinning and Cache Economics

For multi-turn Q&A sessions, session pinning keeps subsequent requests in a session on the same model that served the first request. This prevents a conversation from switching models mid-session and keeps the KV-cache warm for that session's prefix, reducing redundant input token processing.

To pin a session, pass the `X-Model-Affinity` header with a stable session identifier on each request in the session. The first call routes normally through the MoE classifier. Subsequent calls with the same session ID are served by the same model without re-running the classifier. You verify pinning by reading the `x-model-affinity` header echoed in the response and confirming that `response.model` is consistent across calls.

The OpenAI Python SDK does not expose this header natively. Use `extra_headers` to send it and `.with_raw_response` to read the echo:

Copy

```
import os
from openai import OpenAI

client = OpenAI(
    api_key=os.environ["MODEL_ACCESS_KEY"],
    base_url="https://inference.do-ai.run/v1/"
)

session_id = "session-test-001" # use a stable identifier per user session id

raw = client.chat.completions.with_raw_response.create(
    model="router:cost-governance-demo",
    messages=[
        {"role": "user", "content": "How do I update my billing address?"}
    ],
    extra_headers={"X-Model-Affinity": session_id}
)

response = raw.parse()
affinity_echo = raw.headers.get("x-model-affinity")

print(f"Served by: {response.model}")
print(f"Affinity header echoed: {affinity_echo}")
print(f"Tokens: {response.usage.prompt_tokens} in / {response.usage.completion_tokens} out")
```

Output

```
Served by:                anthropic-claude-4.6-sonnet
Affinity header echoed: session-test-001
Tokens:                  15 in / 245 out
```

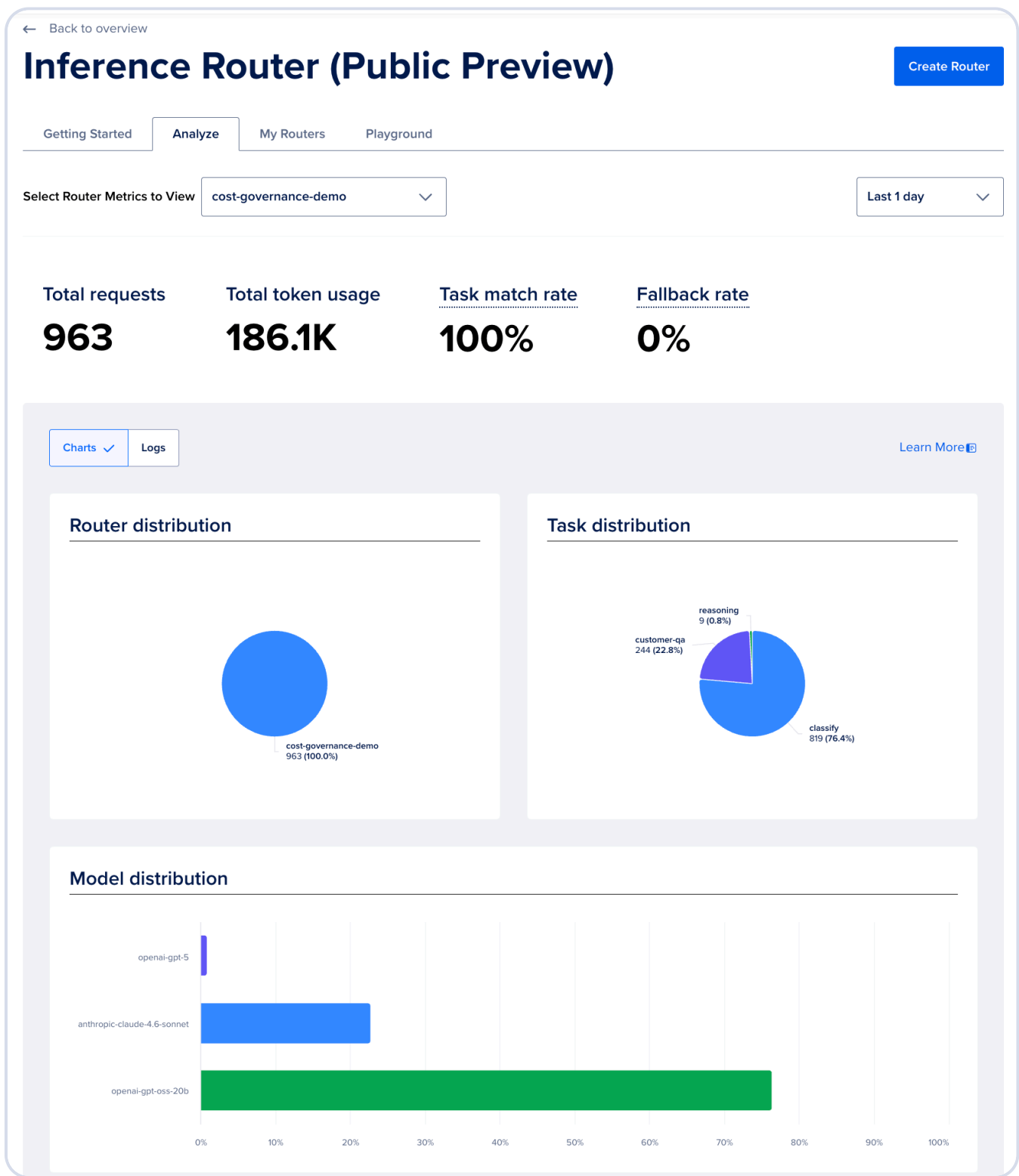
`x-model-affinity: session-test-001` echoed in the response confirms the Router accepted the session identifier. Sending a second request with the same `session_id` returns the same `response.model` value without routing overhead. If the echo is absent, the header was dropped or the endpoint does not support affinity for the matched model.

Prompt caching on Anthropic models yields significant input-cost savings on sessions with long repeated prefixes, since cache reads are billed at approximately 10% of the standard input price. For OpenAI models on DO, automatic prompt caching applies to prompts of 1,024 tokens or more at 50% off the input price. Open-source models on DO do not yet support prompt caching. Routing a session to a different model resets the cache, so session pinning and cache economics are coupled: pinning is the mechanism that keeps the cache active.

Observability and Debugging

The Analyze Dashboard

The Analyze dashboard for the Inference Router is accessible at **AI/ML > Inference > Analyze**. It shows model match rate and fallback rate across your router's traffic. The [features reference](#) documents the available metrics. The [serverless inference metrics reference](#) covers per-request cost and latency signals.



Model match rate is the percentage of requests that matched a named task policy. A rate below 90% usually means task descriptions are too generic or overlap significantly. Fallback rate is the percentage of requests that matched no task and went to `fallback_models`. A high fallback rate means incoming prompts are not represented in your task configuration.

The Playground's Router Evaluation tab provides LLM-as-a-Judge scoring on Completeness, Correctness, Tokens Used, and Latency. Use it to confirm that cheaper-model routing on the classifier path does not degrade output quality relative to a single-model baseline before routing live traffic.

Validating Task Match Quality

To check whether a specific prompt routes to the expected task, send it and inspect `x-model-router-selected-route` :

Copy

```
import os
from openai import OpenAI

client = OpenAI(
    api_key=os.environ["MODEL_ACCESS_KEY"],
    base_url="https://inference.do-ai.run/v1/"
)

raw = client.chat.completions.with_raw_response.create(
    model="router:cost-governance-demo",
    messages=[{"role": "user", "content": "My invoice shows a duplicate charge"}
)

matched_task = raw.headers.get("x-model-router-selected-route")
print(f"Matched task: {matched_task}")
```

Output

Matched task: classify

`classify` is the expected match for a billing complaint. If the header returns `customer-qa` or `fallback` instead, the task descriptions for `classify` and `customer-qa` overlap too much. The classifier matched the Q&A description instead of the `classify` one, or it found no match at all. Make the `classify` description more specific to categorical output and the Q&A description more specific to prose answer output, then re-test with the same prompt.

Common Misconfigurations

Task descriptions that are too generic. A description like “answer questions about the product” overlaps with nearly every prompt in a support backend. Write descriptions that reflect the specific output type: “Answer a customer-facing question about account settings, subscription plans, or billing history, where the response requires retrieving or explaining account-specific information.” Specificity to the output format helps the classifier distinguish tasks reliably.

Task descriptions that overlap. If classify and customer-qa descriptions are semantically similar, the classifier will route ambiguous prompts inconsistently between them. Classify produces a category label; customer-qa produces a prose answer. Describe those different output types in the description, not just the topic area.

Missing `fallback_models` . A router with no `fallback_models` returns an error for unmatched prompts. Configure at least one fallback model, preferably a capable open-source model with low per-request cost.

Pool size exceeding the limit. Each task pool holds a maximum of 3 models. The Router will reject create or edit requests that exceed this limit. See [limits and quotas](#) for current constraints.

Comparing Inference Routing Approaches ^

Rule-Based Routing

Rule-based routing dispatches requests based on structural properties: URL path, a header value, a JSON field, or the value of the `model` parameter itself. It requires no classifier, adds no latency from content evaluation, and is entirely deterministic. The limitation is that it cannot adapt to prompt content. A request with routing logic baked into the URL path cannot change behavior when the prompt content changes, because the dispatch decision was made before the prompt was read.

Rule-based routing is the right starting point when workloads are already segmented by application logic, such as separate endpoints for different task types or an explicit task parameter passed by the client.

Semantic Routing

Semantic routing evaluates prompt content to determine task type before dispatching. The DigitalOcean Inference Router implements semantic routing through its MoE classifier. The classifier adds a small, sub-second routing overhead per request, but it enables dispatch decisions based on actual task content rather than a proxy signal, which means routing is transparent to the application and does not require changes when task patterns evolve.

Research from EMNLP 2024 on the TensorOpera Router found that prompt-content-based routing can achieve a 40% improvement in query efficiency, a 30% cost reduction, and a 10% quality gain relative to single-model deployment at similar cost (Stripelis et al., [TensorOpera Router: A Multi-Model Router for Efficient LLM Inference](#)).

Cost-Aware Dynamic Routing

Cost-aware dynamic routing extends semantic routing with real-time signals: live pricing, current model latency measurements, or per-model quality scores from a continuous evaluation pipeline. The router selects the optimal model at invocation time based on current conditions, not static configuration. This is more flexible but significantly more complex to implement and operate.

For teams running at large scale across multiple providers, cost-aware dynamic routing can recover savings that semantic routing misses when pricing changes or model availability varies. For most SaaS backends on a single provider, semantic routing with well-tuned task descriptions achieves the majority of the cost benefit at much lower operational complexity.

Decision Table: Choosing the Right Strategy

Workload	Recommended Strategy	Reason
Tasks already segmented by endpoint or request field	Rule-based	No classifier overhead; fully deterministic
Mixed-complexity tasks through a single endpoint	Semantic (DO Inference Router)	Per-prompt dispatch; no app code changes required
Multi-provider, high-volume, price-sensitive backend	Cost-aware dynamic	Adapts to real-time pricing and availability changes
Single-model workload, no meaningful task variation	No router	Router adds overhead without routing benefit
Compliance environment requiring auditable model selection	Rule-based or no router	Semantic routing is probabilistic, not fully explainable

When to Use This Pattern and When Not To



Use the Inference Router when:

Your backend handles requests with distinct task-complexity tiers that differ meaningfully in model requirements. A backend where 70% of requests are classification calls and 10% are complex reasoning tasks is a strong fit. Cost savings scale with the proportion of cheap-task traffic, and the Router captures them without modifying application code.

Your backend runs agentic pipelines with multiple sequential model calls at different complexity levels. Each call in the pipeline routes independently based on its prompt content, and session pinning keeps multi-turn agent sessions on the same model for cache consistency. For a deeper look at agentic workload patterns, see [inference routing and model task matching](#).

You want cost governance without routing logic in your application layer. The Router handles dispatch; your application sees a single endpoint and a consistent response format.

Do not use the Inference Router when:

Your workload is uniform in task complexity. A backend where every request requires the same model provides no routing benefit, and the sub-second overhead adds latency with no cost savings.

You are in a compliance or audit environment that requires deterministic, auditable model selection on every request. Semantic routing is probabilistic: classification accuracy is high but not guaranteed, and the dispatched model depends on prompt content in ways that are not fully explainable from the request alone.

The models you need are outside DigitalOcean's catalog. The Inference Router routes to models available within the [Inference Engine](#) only. For workloads that span multiple providers, a self-hosted semantic router or cost-aware dynamic router is the appropriate architecture.

You need a break-even analysis between serverless and dedicated inference before committing to a routing architecture. See [dedicated vs. serverless inference at scale](#) for the capacity and cost crossover points. The data privacy implications of inference routing are covered in the [data privacy documentation](#).

Troubleshooting



Invocation fails with an authentication error despite individually valid credentials.

The MAK (`$MODEL_ACCESS_KEY`) and PAT (`$DIGITALOCEAN_TOKEN`) belong to different DigitalOcean teams. Both must belong to the same team, or invocation fails regardless of router configuration. In the pre-draft verification runs for this tutorial, all early probe failures were caused by this mismatch, not by any Router or GPT-5 limitation. Check your team membership in the DO console. Regenerate credentials under the correct team and store them as the correct environment variables before retrying.

A commercial model (Claude or GPT-5) returns an access error on invocation.

Your account is below Tier 3. Claude Sonnet 4.6 and GPT-5 require Tier 3+ accounts. Tier 1 and Tier 2 accounts are limited to open-source models. [Check your tier](#) and upgrade before retrying. Reconfigure the router with open-source-only pools if an upgrade is not feasible.

The reasoning path returns `"content": null` with `"finish_reason": "length"` .

The `max_completion_tokens` budget is too small. GPT-5 and other reasoning models spend a portion of the token budget on internal reasoning steps before generating the visible answer. Setting `max_completion_tokens: 1024` on the reasoning prompt in this tutorial returned `content: null` in the confirmed live test. The same prompt completed successfully at `max_completion_tokens: 4096` . Set a minimum of 4,096 tokens on any reasoning path, and adjust upward if your prompts expect longer detailed outputs.

The router create call returns HTTP 400 with `"model router name already exists"` .

The name is already taken in your team. The full error shows `"id": "invalid_argument"` and `"message": "rpc error: code = InvalidArgument desc = model router name already exists"` . Choose a different name for the new router.

`x-model-router-selected-route` consistently returns `fallback` .

No task description matched the incoming prompt. Common causes: task descriptions are too generic, descriptions overlap significantly, or the prompt type is not represented in your configuration. Check the Analyze dashboard for fallback rate trends, then make each task description more specific and distinct from the others. Test individual prompts

using the `.with_raw_response` pattern from the “Reading the Cost Signal per Request” section to confirm which task they match before routing live traffic.

Cleaning Up

To delete the router created in this tutorial, send a DELETE request using the UUID from the create response:

Copy

```
curl -s -X DELETE \
  "https://api.digitalocean.com/v2/gen-ai/models/routers/<your-router-uuid>"
-H "Authorization: Bearer $DIGITALOCEAN_TOKEN"
```

During public preview the Router is free, so there is no billing impact from leaving it running. Delete it when you no longer need it to keep your router list clean before the Router reaches GA pricing.

FAQ

What Is the DigitalOcean Inference Router?

An inference router is a middleware component that receives LLM API requests and directs each one to an appropriate model based on the task type and configured selection policy. It sits between the client application and the model-serving layer, enabling dynamic model selection without changes to application code. The DigitalOcean Inference Router uses a semantic MoE classifier to match each prompt against the natural-language task descriptions you define when creating the router.

How Does the DigitalOcean Inference Router Differ from a Standard API Gateway?

A standard API gateway handles authentication, rate limiting, and request forwarding to a fixed backend. The DigitalOcean Inference Router adds semantic task matching: an MoE classifier reads each incoming prompt, matches it against your configured task descriptions, and dispatches it to the appropriate model pool. The selection policy on

that pool then picks the specific model by cost, speed, or explicit ranking, without any routing logic required in your application code.

Can the Inference Router Reduce LLM API Costs?

Yes. By routing simpler tasks to smaller, less expensive models and reserving frontier models for high-complexity tasks, the Router reduces average cost per request across a mixed workload. At a traffic split of 700,000 classify requests, 250,000 Q&A requests, and 50,000 reasoning requests per month, the routed configuration costs \$2,850.30 vs. \$4,716.60 for a Claude Sonnet 4.6 baseline, a 39.6% reduction. A uniform-complexity workload sees minimal savings; backends with a high share of classification or summarization requests see material cost reduction.

What Task-Matching and Selection Policies Does the DigitalOcean Inference Router Support?

The Router supports four selection policies: Cost Efficiency (`prefer: cheapest`), Speed Optimization (`prefer: fastest`), Manual Ranking (models tried in the order you list, no `selection_policy` field required), and Optimal (DO-defined preset task types only, not available for custom tasks). Task matching is semantic: an MoE classifier matches each incoming prompt against the natural-language `custom_task.description` fields you provide when creating the router. There is no ordered rule evaluation. Refer to [Inference Router how-to guide](#) for the current full list of supported parameters.

What Is the Difference Between Serverless Inference and Dedicated Inference in the Context of Routing?

Serverless inference runs on shared infrastructure with per-request billing and automatic scaling, including scale-to-zero. Dedicated inference runs on reserved compute with predictable latency and fixed capacity. The DigitalOcean Inference Router routes requests to both serverless and dedicated inference backends. With Speed Optimization or Manual Ranking selection policies, dedicated models become selectable for workloads requiring guaranteed capacity and consistent latency SLAs. See [the Inference documentation](#) for current configuration options for each deployment type.

Does the Inference Router Support Agentic Workloads?

Yes. DigitalOcean designed the Inference Router as part of the April 2026 Inference Engine launch specifically to support agentic workload scaling. Agentic workloads typically involve multiple sequential model calls with varying complexity, which benefits from per-request routing to appropriately sized model tiers. Use `X-Model-Affinity`

session pinning to keep multi-turn agent sessions on the same model for KV-cache consistency.

How Do I Debug a Router Configuration That Is Not Matching as Expected?

Open the Analyze dashboard and check the model match rate and fallback rate. A high fallback rate means the Router is not matching incoming prompts to any configured task, usually because `custom_task.description` fields are too generic or overlap too much with each other. Make each task description narrower and more distinct, then re-test using the `x-model-router-selected-route` header on individual prompts. Also confirm that the `MODEL_ACCESS_KEY` used for invocation belongs to the same DO team as the PAT used to create the router; a team mismatch causes invocation failure regardless of routing configuration.

Is the DigitalOcean Inference Router Compatible with the OpenAI API Format?

Yes. The Router is invoked via the OpenAI-compatible endpoint at `https://inference.do-ai.run/v1/chat/completions`. Set `"model": "router:<your-router-name>"` in the request body. The response follows the standard OpenAI chat completions format, with the addition of the `x-model-router-selected-route` header indicating which task policy matched. Confirm current compatibility details at [Inference Router how-to guide](#).

Does GPT-5 Work Through the Inference Router over Chat Completions?

Yes, confirmed. Despite GPT-5 requiring the Responses API when called directly on OpenAI's platform, the DigitalOcean Inference Router serves GPT-5 transparently over `/v1/chat/completions`. No special handling is required in the client application. GPT-5 may answer correctly while declining to expose internal reasoning steps; do not write application logic that depends on chain-of-thought output being present in the response body.

What Account Tier Is Required to Use Claude or GPT-5 in a Router Policy?

Commercial models from Anthropic and OpenAI require a Tier 3 or higher [DigitalOcean account](#). Tier 1 and Tier 2 accounts have access to open-source models only. Verify

your tier in the DO console before configuring router task policies that include Claude or GPT-5 model slugs.

Conclusion

This tutorial built a three-path Inference Router for a SaaS support backend, configured task policies using Cost Efficiency, Manual Ranking, and single-model selection, invoked each path through the OpenAI-compatible chat completions endpoint, and read per-request cost signals from the response header and model field. The monthly cost comparison shows a 39.6% savings against a Claude Sonnet 4.6 baseline and 63.7% against Claude Opus 4.7 at a traffic split of 700,000 classify, 250,000 Q&A, and 50,000 reasoning requests per month. The confirmed per-request costs from the June 16, 2026 live runs are \$0.00004070 on the classifier path, \$0.00445200 on Q&A, and \$0.03417625 on reasoning.

With this router in place, you can govern LLM inference spend at the model selection layer without modifying application code, read the dispatched model and matched task on every response for per-request cost attribution, and apply session pinning on the Q&A path to keep KV-cache warm across multi-turn conversations. Adding coverage for new task types requires only a new task policy with a well-written description; the application endpoint stays the same.

For a deeper look at how to choose which metrics to measure across your inference stack before configuring model tiers, see [Metrics that Matter with Serverless Inference](#). For workloads that require predictable latency SLAs at fixed capacity rather than serverless autoscaling, see [dedicated vs. serverless inference at scale](#). For the sibling tutorial on building a cost-aware AI support API with the Inference Router, see [Cost-Aware AI Support API with the Inference Router](#). The Inference Router is in public preview at time of writing; check the [Inference Engine product page](#) for GA status.

Thanks for learning with the DigitalOcean Community. Check out our offerings for compute, storage, networking, and managed databases.

Learn more about our products →

About the author



Vinayak Baranwal Author

Technical Writer II

[See author profile](#)

Building future-ready infrastructure with Linux, Cloud, and DevOps. Full Stack Developer & System Administrator. Technical Writer @ DigitalOcean | GitHub Contributor | Passionate about Docker, PostgreSQL, and Open Source | Exploring NLP & AI-TensorFlow | Nailed over 50+ deployments across production environments.

Category: Tutorial



Tags: AI/ML AI Inference Dedicated-Inference

Still looking for an answer?

[Ask a question](#)

[Search for more help](#)

Was this helpful?

[Yes](#)

[No](#)

Comments(0)

Follow-up questions(0)

B *I* U

Leave a comment...

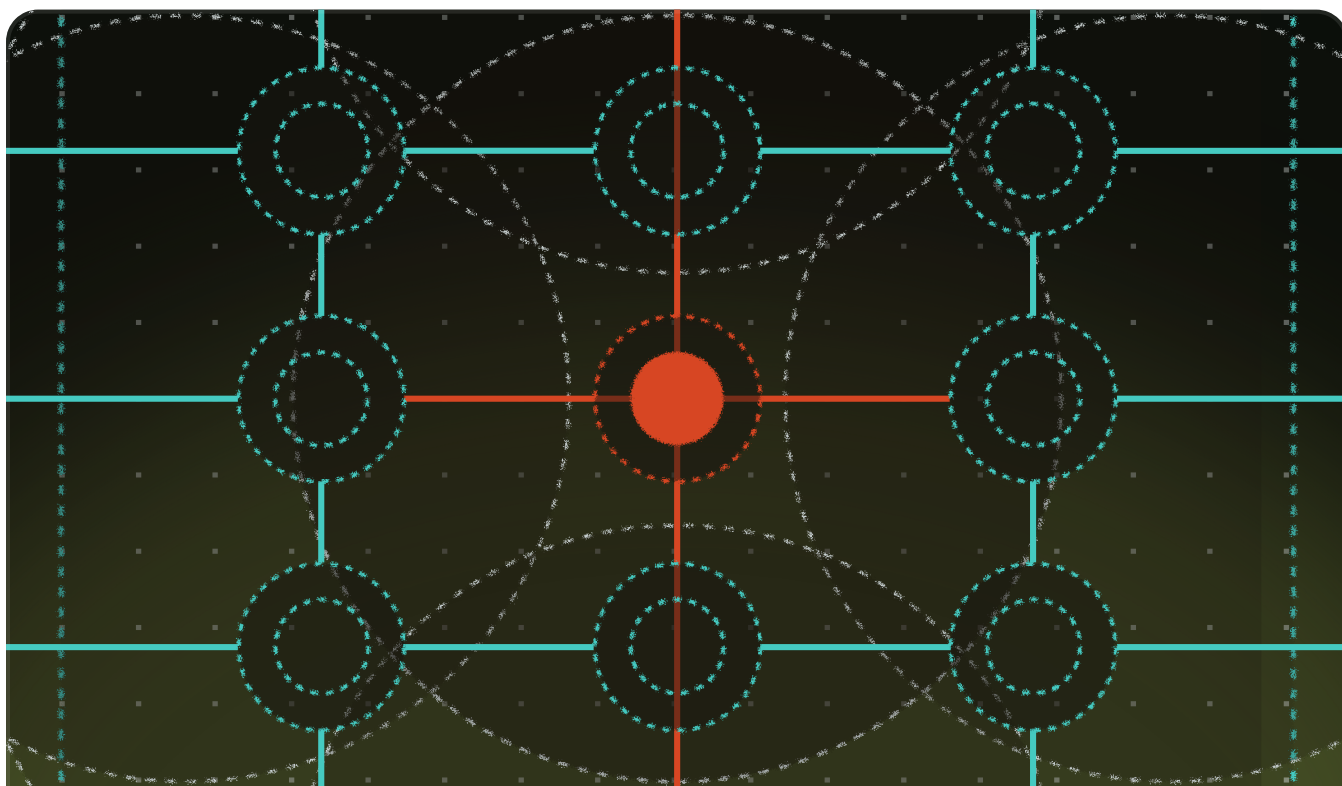
This textbox defaults to using Markdown to format your answer.

You can type `!ref` in this text area to quickly search our full set of tutorials, documentation & marketplace offerings and insert the link!

Sign in/up to comment



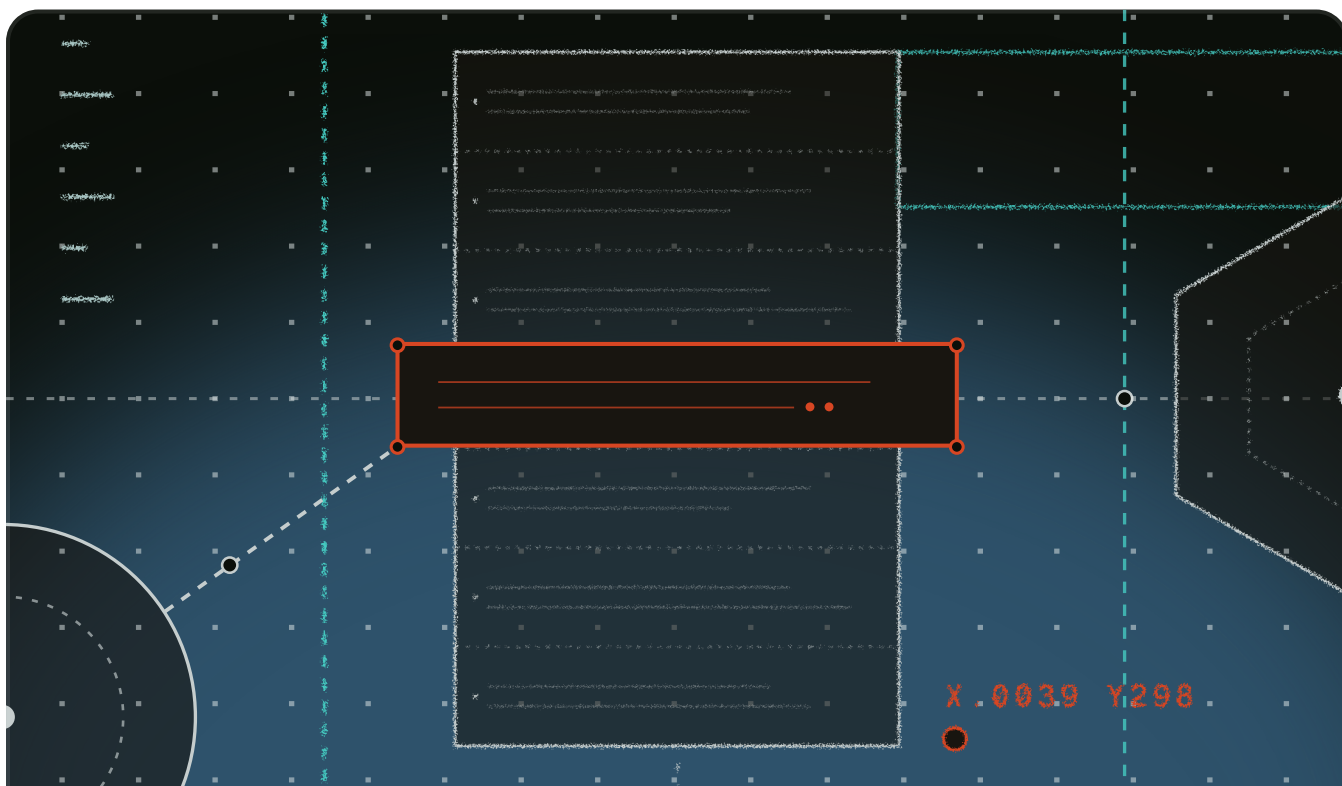
This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.



Станьте участником сообщества

Получайте деньги за написание технических руководств и выберите благотворительную организацию, связанную с технологиями, чтобы получить аналогичную сумму в качестве пожертвования.

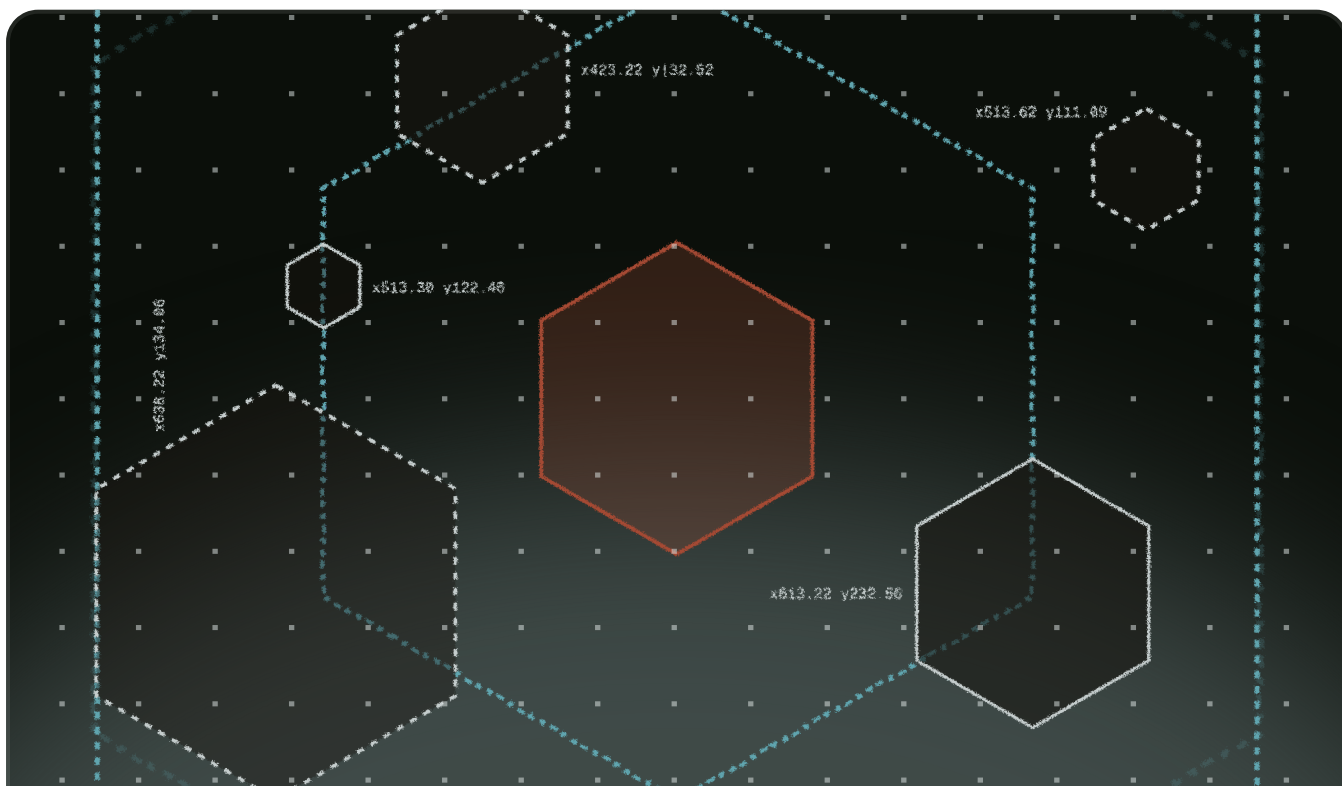
Регистрация →



Документация DigitalOcean

Полная документация по каждому продукту DigitalOcean.

Узнать больше →



Ресурсы для стартапов и компаний, использующих искусственный интеллект

В The Wave есть все, что вам нужно знать о построении бизнеса: от привлечения финансирования до маркетинга вашего продукта.

Узнать больше →

Будьте в курсе событий, подпишитесь на нашу рассылку

Электронная почта *

Отправить

Облако для разработчиков

Масштабируйтесь по мере роста — независимо от того, используете ли вы одну виртуальную машину или десять тысяч.

Просмотреть все продукты

Начните создавать уже сегодня

От логического вывода на базе графического процессора и Kubernetes до управляемых баз данных и хранилищ — у нас есть все необходимое для создания,

масштабирования и развертывания интеллектуальных приложений.

Регистрация

Company



Products



Resources



Solutions



Contact



© 2026 DigitalOcean, LLC. [Sitemap](#). [Cookie Preferences](#)

